

CRT PROJECT REPORT

On

PYTHON PASSWORD SAFE

Submitted by

H.V.SAI PRAVALLIKA

24AK5A0285

R. THANUJA

24AK5A0279

M.GOWRI PRIYA

24AK5A0221



DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING

ANNAMACHARYA INSTITUTE OF TECHNOLOGY & SCIENCES

TIRUPATI

(AUTONOMOUS)

Approved by AICTE, New Delhi & Permanent Affiliation to JNTUA, Anantapuramu. Three B. Tech Programmes (EEE, CIVIL, ECE& CSE) are accredited by NBA, New Delhi. Accredited by NAAC with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA. A-grade awarded by AP Knowledge Mission. Recognized under sections 2(f) 12(B) of UGC Act 1956. Venkatapuram(V), Renigunta(M), Tirupati, Tirupati District, Andhra Pradesh – 517520.

(2026-2027)

CERTIFICATE

This is to certify that the project titled “**Password Safe Generator by using Python**” submitted by **H.V.SAI PRAVALLIKA, R. THANUJA, M.GOWRIPRIYA** from **Electrical & Electronics Engineering** in partial fulfilment of the requirements for the award of the DEGREE of **BACHELOR OF TECHNOLOGY** in is a Bonafide work carried out under our supervision.

Signature of Project Guide

Signature of HOD

DECLARATION

We, hereby declare that this Project report titled “**Password Safe Generator by using Python**” has been written by me. The work carried out is original and has not been submitted to any other University or Institution for the award of the credits.

H.V.SAI PRAVALLIKA

24AK5A0285

R. THANUJA

24AK5A0279

M.GOWRI PRIYA

24AK5A0221

ACKNOWLEDGEMENT

We express our sincere gratitude to Annamacharya Institute of Technology and Sciences, Tirupati, for providing me with the opportunity and facilities to carry out this Mini Project titled “Password Safe Using Python.”

We extend our deep sense of gratitude to my project guide and all the faculty members for their guidance, constructive feedback, and constant motivation during the development of this project.

Finally, we express my sincere thanks to my parents and family members for their unconditional encouragement, inspiration, and moral support, which helped me successfully complete this project.

we acknowledge with gratitude all those who directly or indirectly contributed to the successful completion of this mini project.

CONTENTS:

1. ABSTRACT
2. CONTENTS
3. CHAPTER 1 - Introduction
4. CHAPTER 2 - Literature Survey
5. CHAPTER 3 - System Analysis
6. CHAPTER 4 - System Design
7. CHAPTER 5 - Methodology
8. CHAPTER 6 - Implementation
9. CHAPTER 7 - Testing & Results
- 10.CHAPTER 8 - Advantages, Limitations
- 11.CHAPTER 9 - Future Scope
12. CHAPTER 10 - Conclusion

ABSTRACT

Password management is an important aspect of digital security. Users often maintain multiple accounts and face difficulty remembering passwords. This project develops a Password Safe application using Python that stores, retrieves, and manages passwords efficiently. The system uses file handling techniques to store account credentials. It demonstrates the practical implementation of Python programming concepts such as functions, loops, conditional statements, and file operations.

Keywords: Python, Password Management, Security, File Handling, Data Storage.

CHAPTER-1

INTRODUCTION

1.1 Overview

In today's digital era, the use of online services has increased significantly. Individuals use multiple websites and applications for communication, banking, education, shopping, entertainment, and social networking. Each online platform requires authentication through a username and password to ensure security and privacy. As the number of online accounts increases, users often find it difficult to remember different passwords for different services.

Many users tend to use the same password for multiple accounts or choose weak passwords that are easy to remember. Such practices increase the risk of unauthorized access and cyber-attacks. Therefore, there is a growing need for secure and efficient password management systems that help users store and manage their credentials safely.

The "Password Safe Using Python" project is designed to provide a simple and effective solution for storing and retrieving passwords. This application allows users to save account credentials and access them whenever required. The project demonstrates the practical implementation of Python programming concepts such as functions, file handling, loops, and conditional statements.

1.2 Background

Passwords serve as the primary line of defense in most computer systems and online applications. They help protect sensitive information from unauthorized users. With the rapid growth of internet-based services, individuals are required to maintain multiple accounts, each with unique login credentials. Managing numerous passwords manually can be challenging. Traditional methods such as writing passwords in notebooks or storing them in unsecured files are not reliable and may compromise security. Modern password managers provide a systematic approach to storing and retrieving credentials securely.

The Password Safe project focuses on creating a lightweight password management application that stores account information using Python file-handling techniques. It serves as a learning platform for understanding software development and cybersecurity fundamentals.

1.3 Problem Statement

Many users struggle to remember passwords for various online accounts. As a result, they often use weak passwords or reuse the same password across multiple platforms. This practice can lead to security vulnerabilities and unauthorized access to personal information.

Existing password management solutions may be expensive, complex, or require internet connectivity. Therefore, there is a need for a simple, user-friendly, and locally accessible password management system that can securely store and retrieve passwords.

The proposed Password Safe application addresses this problem by providing an easy-to-use interface for managing account credentials efficiently.

1.4 Objectives of the Project

The main objectives of the Password Safe project are:

- To develop a password management system using Python.
- To store user account credentials securely.
- To provide an option for viewing saved passwords.
- To enable users to search for stored account credentials.
- To demonstrate the use of Python programming concepts such as file handling and functions.
- To create a simple and user-friendly application for educational purposes.

1.5 Significance of the Project:

Password security is an essential aspect of information security. This project highlights the importance of managing credentials effectively and securely. It provides users with a centralized location for storing passwords and reduces the likelihood of forgetting important login information.

From an academic perspective, the project helps students gain practical knowledge of Python programming, software design, file handling, debugging, and testing. It also introduces basic concepts of cybersecurity and authentication systems.

Need for Password Management

Traditional methods of storing passwords are insecure. Password managers help users store credentials safely and retrieve them whenever required.

CHAPTER-2

Literature Survey

2.1 Introduction

A literature survey is an essential part of any project as it provides an understanding of existing systems, technologies, methodologies, and research work related to the proposed project. It helps identify the strengths and limitations of current solutions and provides a foundation for developing an improved system.

In the modern digital world, users access multiple online services such as email accounts, banking applications, educational portals, social media platforms, and e-commerce websites. Each service requires authentication through usernames and passwords. Managing numerous passwords has become a significant challenge for users, leading to the development of password management systems.

Password managers are applications designed to securely store and manage passwords. They help users generate strong passwords, organize credentials, and reduce the risk of password-related security breaches. This chapter reviews various password storage techniques, password management systems, and existing technologies related to password security.

2.2 Traditional Password Management Methods

Before the development of digital password management systems, users relied on traditional methods to store and remember passwords.

2.2.1 Manual Storage

Many users write their passwords in notebooks, diaries, or personal notes. This method is simple and does not require any technical knowledge. However, it presents several security risks.

Advantages:

- Easy to implement.
- No software requirements.

Disadvantages:

- Passwords can be lost or stolen.
- Difficult to manage a large number of accounts.

2.2.2 Memorization Techniques

Some users attempt to remember all their passwords without storing them anywhere. This method avoids the risk of physical theft but introduces other challenges.

Advantages

- No external storage required.
- Reduced risk of physical password exposure.

Disadvantages

- Difficult to remember multiple passwords.
- Users often choose weak or repeated passwords.

2.3 Browser-Based Password Managers

Modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari provide built-in password management functionality. These browsers can save login credentials and automatically fill them when users visit websites.

2.3.1 Features

- Automatic password storage.
- Auto-fill functionality.
- Password synchronization across devices.

Advantages

- Convenient and easy to use.
- Integrated directly into browsers.

Limitations

- Browser-specific storage.
- Security depends on browser protection mechanisms.

2.4 Dedicated Password Manager Applications

Dedicated password managers are specialized software applications designed specifically for secure password storage and management. Examples include commercial and open-source password management tools. These applications typically use encryption algorithms to protect stored credentials and offer advanced features such as password generation, synchronization, and backup.

Features

- Encrypted password storage.
- Strong password generation.

Advantages

- Enhanced security.
- Centralized credential management.
- Improved user experience.

Disadvantages

- Some applications require subscriptions.
- Dependence on third-party services.
- Additional learning curve for users.

2.5 File-Based Password Storage Systems

File-based password management systems store credentials in local files such as text files, CSV files, or structured data formats. These systems are commonly used in educational and mini-project applications.

In such systems, passwords are stored and retrieved using programming languages such as Python. File handling techniques enable users to add, view, and search credentials efficiently.

Advantages

- 1.Simple implementation.
- 2.Easy to understand.
- 3.Suitable for academic projects.
- 4.No internet connection required.

Disadvantages

- 1.Limited security.
- 2.Lack of encryption.

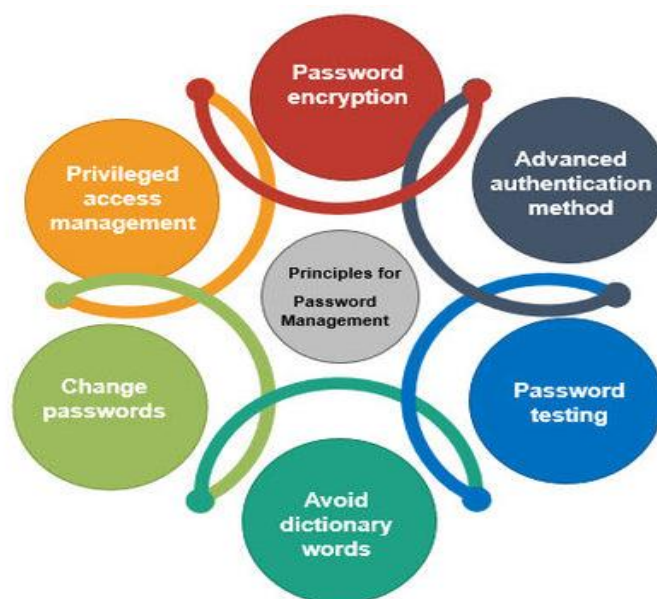


Fig: Password Management

CHAPTER -3

SYSTEM ANALYSIS

3.1 Introduction

System Analysis is an important phase in software development that involves studying the existing system, identifying its limitations, and proposing an improved solution. It helps in understanding user requirements and designing a system that effectively addresses the identified problems.

The Password Safe project aims to provide a simple and efficient solution for storing and managing passwords. This chapter discusses the existing system, its drawbacks, the proposed system, and the functional and non-functional requirements of the project.

3.2 Existing System

Traditionally, users manage their passwords using notebooks, diaries, spreadsheets, or memory-based techniques. Some users also depend on browser-based password managers for storing credentials.

In manual systems, passwords are written on paper or stored in unsecured files. Although these methods are simple, they are not suitable for managing multiple accounts securely. Users often forget passwords or use weak passwords to make them easier to remember.

Browser-based password managers provide automatic password storage and retrieval. However, they are limited to specific browsers and may not provide complete control over password management.

3.2.1 Characteristics of Existing Systems

- Passwords are stored manually or remembered by users.
- Browser password managers are browser-dependent.
- Limited accessibility and flexibility.
- Increased risk of password loss or theft.

3.2.2 Limitations of Existing System

The existing password management methods suffer from several disadvantages.

1. Password Forgetting: Users often maintain numerous online accounts. Remembering different passwords for each account becomes difficult, leading to forgotten credentials.
2. Security Risks: Writing passwords on paper or storing them in unsecured files can expose sensitive information to unauthorized users.

3. Password Reuse: To avoid forgetting passwords, users frequently reuse the same password across multiple websites. This increases security risks.

4. Lack of Centralized Management: Passwords stored in different locations are difficult to manage and organize efficiently.

5. Limited Learning Value: Traditional methods do not provide opportunities for understanding software development concepts or password management techniques.

⟨ These limitations create the need for a better password management system.

3.3 Proposed System

The proposed Password Safe system is a Python-based application designed to store, view, and search passwords efficiently. The system provides a centralized platform for managing account credentials using file handling techniques.

The application allows users to save passwords associated with different accounts and retrieve them whenever required. The system is lightweight, user-friendly, and suitable for educational purposes.

3.3.1 Features of Proposed System

- Add new passwords.
- View stored passwords.
- Search passwords by account name.
- Store data using files.
- User-friendly menu-driven interface.

3.3.2 Advantages of Proposed System

The proposed Password Safe application offers several advantages over traditional methods.

1. Simplicity: The application is easy to use and requires minimal technical knowledge.
2. Centralized Storage: All passwords are stored in a single location, making management easier.
3. Easy Retrieval: Users can quickly search and retrieve account credentials.
4. Offline Accessibility: The system does not require an internet connection to function.
5. Educational Value: The project demonstrates practical implementation of Python programming concepts such as functions, loops, file handling, and conditional statements.

EXISTING SYSTEM AND COMPARISON

Existing System

- ▶ Existing system is a static website so you have to use file system to save record.
- ▶ Time consuming system.
- ▶ No proper system to make announcements.
- ▶ Important information can be missed while writing on page
- ▶ Inefficient and slow search.

Proposed System

- ▶ New proposed system allow to user to save record is database.
- ▶ Fast and easy to use
- ▶ An easy way to make a announcements.
- ▶ Full fledge information system.
- ▶ Efficient and proper availability of data.

Fig: Comparison

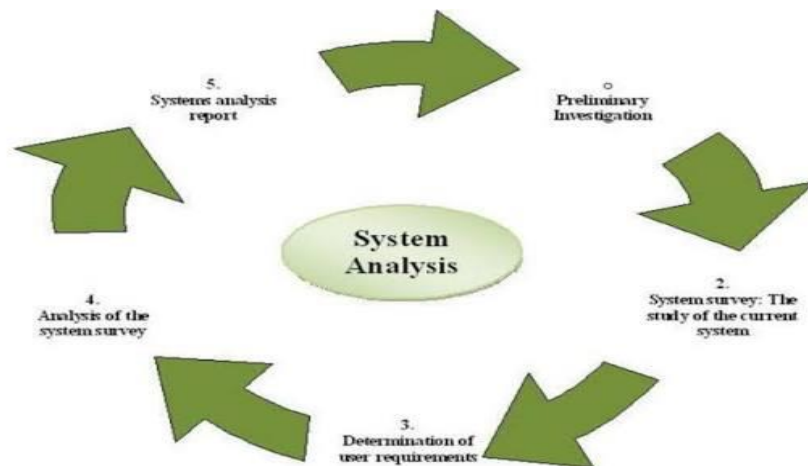


Fig: System Analysis

CHAPTER-4

SYSTEM DESIGN

4.1 Introduction

System Design is the process of defining the architecture, modules, interfaces, and data flow of a software system. It acts as a blueprint for the development of the application and ensures that the system meets the specified requirements effectively.

The proposed Password Safe Using Python is designed as a simple menu-driven application that allows users to store, view, and search passwords. The system follows a straightforward architecture where user inputs are processed through Python functions and the data is stored in a text file.

4.2 System Architecture

The Password Safe application consists of three main components:

User Interface – Allows users to interact with the system through menu options.

Application Logic – Processes user requests using Python functions such as Add Password, View Password, and Search Password.

Data Storage – Stores account credentials in a text file for future retrieval.

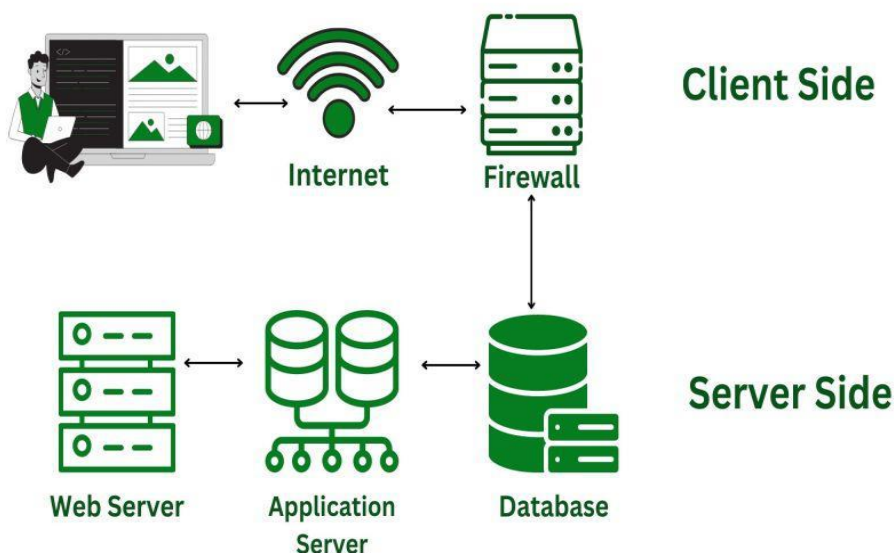


Fig: System Architecture

4.3 Module Design

The system is divided into four modules:

1. Add Password Module
2. View Password Module
3. Search Password Module
4. Exit Module

Each module performs a specific function and contributes to the overall operation of the Password Safe application.

The simple design ensures ease of use, maintainability, and efficient password management while demonstrating Python programming concepts such as file handling, functions, and conditional statements.

Log in

Username *

Enter your Playground username.

Password *

Enter the password that accompanies your username.

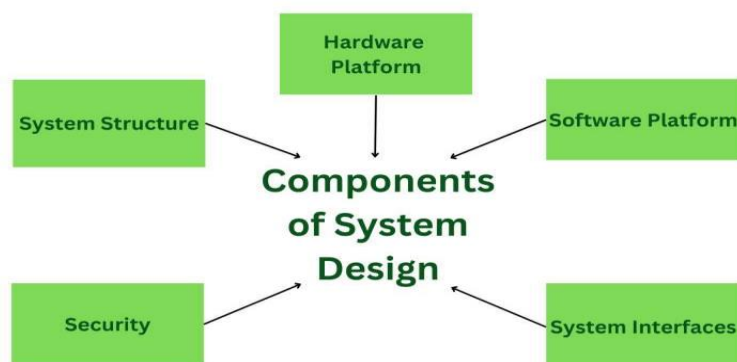


Fig: Components of System Design

CHAPTER 5

METHODOLOGY

5.1 Introduction

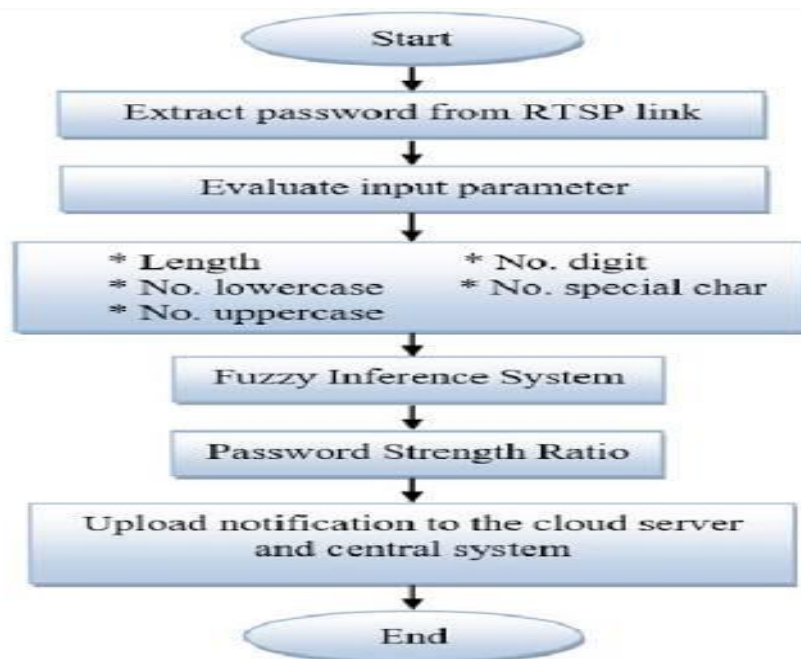
The methodology describes the systematic approach followed in the development of the Password Safe Using Python project. The project is designed to provide a simple and efficient password management system that allows users to store, view, and search passwords. The development process includes requirement analysis, system design, coding, testing, and implementation.

5.2 Development Methodology

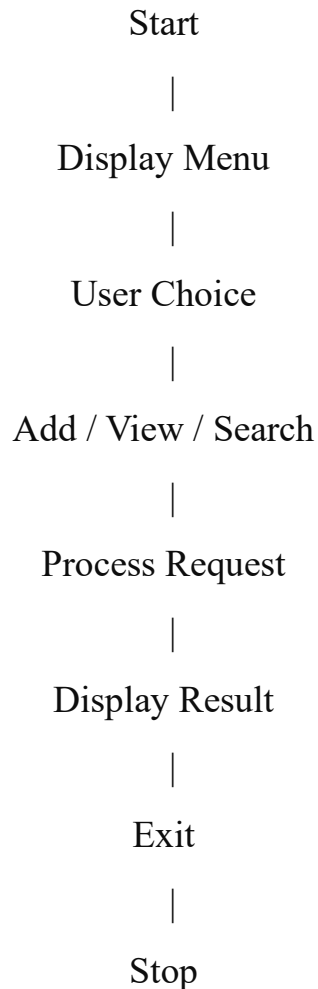
The Password Safe application follows a menu-driven approach. Initially, the user selects an operation from the main menu. Based on the user's choice, the corresponding function is executed.

The methodology consists of the following steps:

1. Display the menu options to the user.
2. Accept the user's choice.
3. If the user selects Add Password, the account name and password are stored in a text file.
4. If the user selects View Passwords, all stored credentials are displayed.
5. If the user selects Search Password, the system searches for the specified account and displays the corresponding password.



5.3 Flow of the System:



This methodology ensures efficient password management while demonstrating Python concepts such as functions, loops, conditional statements, and file handling.

Requirements Specification:

5.4 Introduction

Requirements Specification defines the hardware and software resources needed for the successful development and execution of the Password Safe application. It also describes the functional and non-functional requirements of the system.

5.5 Hardware Requirements

The hardware requirements for the Password Safe project are minimal since the application is lightweight and developed using Python.

- Component
- Specification
- Processor
- Intel Core i3 or above
- RAM
- 4 GB Minimum

- Hard Disk
- 100 MB Free Space
- Monitor
- Standard Display
- Keyboard & Mouse
- Required

5.6 Software Requirements

The software requirements necessary for developing and running the Password Safe application are listed below.

- Software
- Specification
- Operating System
- Windows 10/11, Linux, MacOS
- Programming Language
- Python 3.x
- IDE
- VS Code / PyCharm / IDLE
- Storage Method
- Text File (.txt)



Fig: Hardware And Software

CHAPTER -6

IMPLEMENTATION

6.1 Introduction

Implementation is the phase in which the proposed system design is converted into a working software application. The Password Safe application is developed using Python programming language and utilizes file handling techniques to store and manage user credentials. The implementation phase focuses on coding, testing, debugging, and integrating different modules of the system.

The primary objective of implementation is to provide a simple and efficient password management solution that enables users to store, retrieve, and search passwords easily.

6.2 Programming Language Used

Python is used for the implementation of the Password Safe project because of its simplicity, readability, and extensive support for file handling operations.

Advantages of Python

- Easy to learn and use.
- Supports rapid application development.
- Provides built-in file handling functions.
- Open-source programming language.

These features make Python suitable for developing small-scale applications such as the Password Safe system.

6.3 Implementation of Modules:

The Password Safe application consists of four main modules:

- 1) **Add Password Module:** This module allows users to add account credentials to the system.

Process:

- The user enters the account name.
- The user enters the corresponding password.
- The application stores the information in a text file.

- A success message is displayed.

This module ensures that new account credentials can be stored efficiently for future use.

Add Password Function:

```
def add_password ():
```

```
    account = input ("Enter Account Name: ")  
    password = input ("Enter Password: ")  
    with open ("passwords.txt", "a") as file:  
        file. Write (account + " | " + password + "\n")  
    print ("Password Saved Successfully!")
```

1) **View Password Module:** This module displays all stored account credentials.

Process:

- The system reads the contents of the password file.
- Stored records are displayed on the screen.
- Users can view all saved passwords.

This functionality helps users access their stored credentials whenever required.

View Password Function:

```
def view_passwords ():
```

```
    with open ("passwords.txt", "r") as file:  
        print (file. read ())
```

3) **Search Password Module:** This module enables users to search for a specific account.

Process:

- User enters the account name.
- The application searches the file.
- If the account exists, the password is displayed.
- Otherwise, an error message is shown.

This feature improves accessibility and saves time when locating credentials.

Search Password Function:

```
def search_password ():  
    account_name = input ("Enter Account Name: ")  
    with open ("passwords.txt", "r") as file:  
        for line in file:  
            account, password = line. strip (). split (" | ")  
            if account == account_name:  
                print ("Password:", password)
```

4) Exit Module: This module terminates the execution of the application safely.

Process:

- User selects Exit. And Program execution stops.
- Application closes successfully.

Main Program:

```
while True:  
    print ("\nPASSWORD SAFE")  
    print ("1. Add Password")  
    print ("2. View Passwords")  
    print ("3. Search Password")  
    print ("4. Exit")  
    choice = input ("Enter Choice: ")  
    if choice == "1":  
        add_password ()  
    elif choice == "2":  
        view_passwords ()  
    elif choice == "3":  
        search_password ()  
    elif choice == "4":  
        break
```

6.4 Algorithm

Step 1: Start the program.

Step 2: Display menu options.

Step 3: Read user choice.

Step 4: If choice is Add Password:

Enter account name.

Enter password.

Save details to file.

Step 5: If choice is View Passwords:

Read data from file.

Display stored records.

Step 6: If choice is Search Password:

Enter account name.

Search file and display result.

Step 7: Repeat until Exit is selected.

Step 8: Stop.

6.5 APPENDIX -A

SOURCE CODE:

```
import random

import string

print ("Welcome to Password Generator!")

n_letters = int (input ("How many letters do you want? "))

n_numbers = int (input ("How many numbers do you want? "))

n_symbols = int (input ("How many symbols do you want? "))

password_list = []

for _ in range(n_letters):

    password_list.append (random. Choice (string. ascii_letters))

for _ in range(n_numbers):

    password_list.append (random. Choice (string. Digits))

for _ in range(n_symbols):

    password_list.append (random. Choice ("! @#$%^&*()_+"))

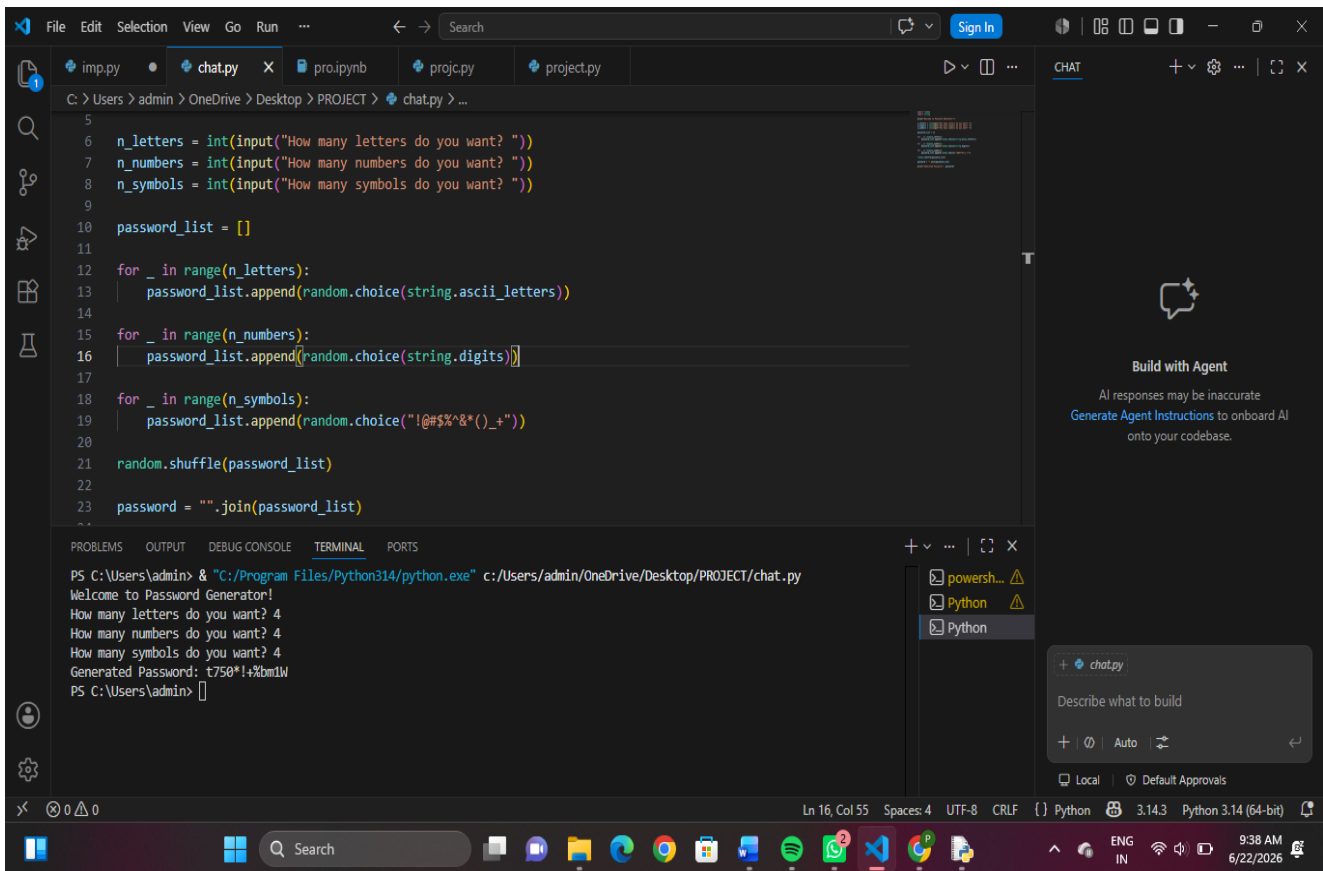
random. Shuffle(password_list)

password = "".join(password_list)

print ("Generated Password:", password)
```

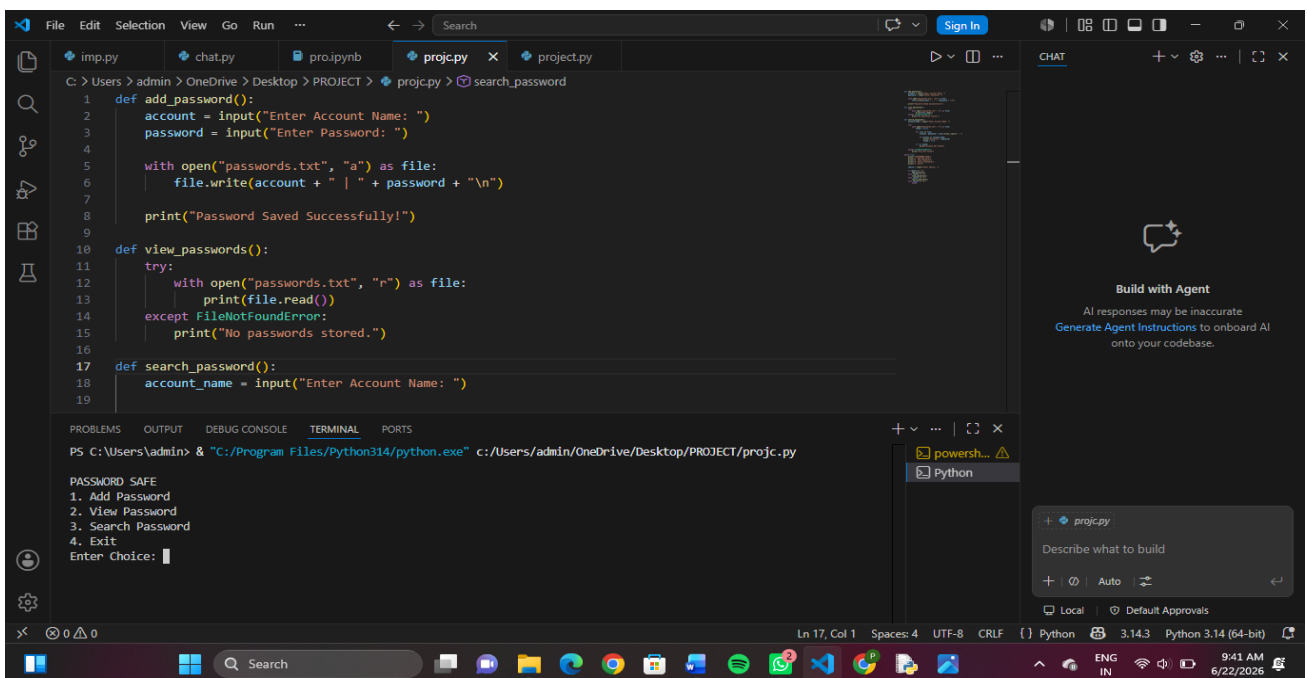

6.6 APPENDIX -B

OUTPUT & SCREENSHOTS:



```
File Edit Selection View Go Run ... Search Sign In
C: > Users > admin > OneDrive > Desktop > PROJECT > chat.py > ...
5
6 n_letters = int(input("How many letters do you want? "))
7 n_numbers = int(input("How many numbers do you want? "))
8 n_symbols = int(input("How many symbols do you want? "))
9
10 password_list = []
11
12 for _ in range(n_letters):
13     password_list.append(random.choice(string.ascii_letters))
14
15 for _ in range(n_numbers):
16     password_list.append(random.choice(string.digits))
17
18 for _ in range(n_symbols):
19     password_list.append(random.choice("!@#$%^&*()_+"))
20
21 random.shuffle(password_list)
22
23 password = "".join(password_list)
24
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin> & "C:/Program Files/Python314/python.exe" c:/Users/admin/OneDrive/Desktop/PROJECT/chat.py
Welcome to Password Generator!
How many letters do you want? 4
How many numbers do you want? 4
How many symbols do you want? 4
Generated Password: t750*!+%bm1W
PS C:\Users\admin>
```

Fig: OUTPUT 1



```
File Edit Selection View Go Run ... Search Sign In
C: > Users > admin > OneDrive > Desktop > PROJECT > projc.py > search_password
1 def add_password():
2     account = input("Enter Account Name: ")
3     password = input("Enter Password: ")
4
5     with open("passwords.txt", "a") as file:
6         file.write(account + " | " + password + "\n")
7
8     print("Password Saved Successfully!")
9
10 def view_passwords():
11     try:
12         with open("passwords.txt", "r") as file:
13             print(file.read())
14     except FileNotFoundError:
15         print("No passwords stored.")
16
17 def search_password():
18     account_name = input("Enter Account Name: ")
19
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin> & "C:/Program Files/Python314/python.exe" c:/Users/admin/OneDrive/Desktop/PROJECT/projc.py
PASSWORD SAFE
1. Add Password
2. View Password
3. Search Password
4. Exit
Enter Choice: 1
```

Fig: OUTPUT2

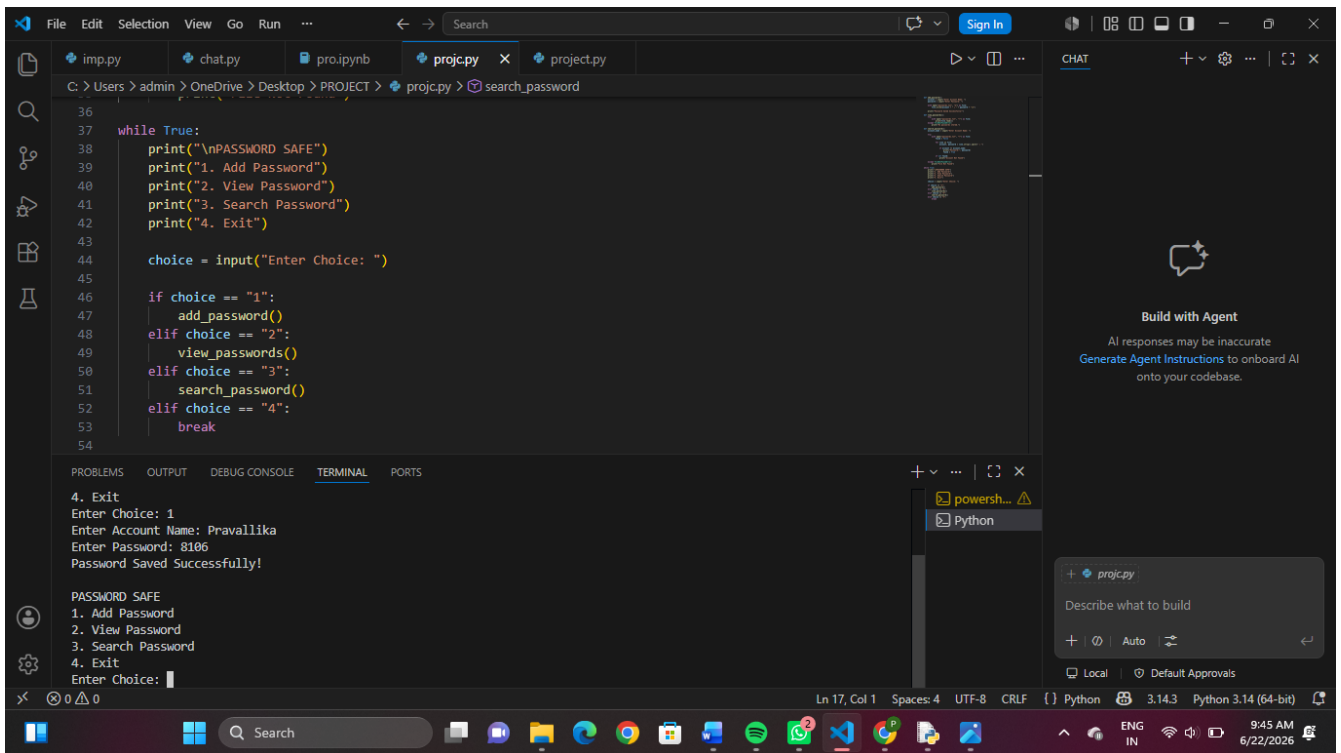


Fig: OUTPUT3

CHAPTER -7

TESTING AND RESULTS

7.1 Introduction

Testing is an important phase in software development that ensures the system functions correctly according to the specified requirements. The main objective of testing is to identify errors, verify system functionality, and improve reliability.

The Password Safe application was tested using different input conditions to verify the proper functioning of all modules. The testing process confirmed that the application successfully stores, retrieves, and searches passwords.

7.2 Testing Methodology

The testing of the Password Safe application was performed using functional testing techniques. Each module was tested individually to ensure that it produced the expected results.

The following modules were tested:

- Add Password Module
- View Password Module
- Search Password Module
- Exit Module

The outputs obtained were compared with the expected outputs to verify correctness.

7.3 Results

The Password Safe application was executed successfully and all modules worked as expected. The system was able to store account credentials, display saved records, search for passwords, and terminate the application correctly.

Sample Output 1

PASSWORD SAFE

- | | |
|--------------------|-------------------|
| 1. Add Password | 2. View Passwords |
| 3. Search Password | 4. Exit |

Enter Choice: 1

Enter Account Name: Gmail

Enter Password: gmail123

Password Saved Successfully!

Sample Output 2

Enter Choice: 2

Stored Passwords:

Gmail | gmail123

Instagram | insta456

Facebook | fb789

7.4 Discussion

The testing results indicate that the Password Safe application performs all intended operations successfully. The system demonstrates proper implementation of file handling, searching mechanisms, and user interaction. The application provides a simple and effective solution for password management while serving as an educational project for learning Python programming concepts.

CHAPTER-8

Advantages, Limitations

ADVANTAGES AND LIMITATIONS

8.1 Advantages

The Password Safe application provides several benefits to users and developers. The major advantages are:

1. **Easy to Use:** The application provides a simple menu-driven interface that can be easily operated by users with basic computer knowledge.
2. **Centralized Password Storage:** All account credentials are stored in a single location, making password management easier and more organized.
3. **Quick Password Retrieval:** Users can search and retrieve stored passwords quickly without remembering multiple credentials.
4. **Lightweight Application:** The system requires minimal hardware and software resources, making it suitable for almost any computer.
5. **Educational Value:** The project helps students understand Python programming concepts such as functions, loops, conditional statements, and file handling.

8.2 Limitations

Although the Password Safe application provides useful functionality, it has certain limitations.

1. **No Password Encryption:** Passwords are stored in plain text format, which may expose sensitive information if the file is accessed by unauthorized users.
2. **Limited Security Features:** The system does not provide advanced security mechanisms such as multi-factor authentication or biometric verification.
3. **Local Storage Only:** Passwords are stored on a single device and cannot be accessed from other devices.
4. **Limited Scalability:** The current implementation is designed for educational purposes and may not support enterprise-level password management requirements.

CHAPTER-9

FUTURE SCOPE

9.1 Future Scope

The current Password Safe application successfully stores, retrieves, and manages passwords using Python file handling techniques. However, there are several opportunities for further enhancement to improve the functionality, security, and usability of the system.

1. **Password Encryption:** In the present system, passwords are stored in plain text format. Future versions can implement encryption algorithms such as AES or SHA to enhance security and protect sensitive information from unauthorized access.
2. **Graphical User Interface (GUI):** The current application uses a console-based interface. A graphical user interface can be developed using Python libraries such as Tkinter or PyQt to provide a more attractive and user-friendly experience.
3. **Database Integration:** Instead of storing data in text files, future implementations can use databases such as MySQL, SQLite, or PostgreSQL for efficient data management and improved scalability.
4. **User Authentication:** A login system can be added to ensure that only authorized users can access stored passwords. This feature will provide an additional layer of security.
5. **Password Generator:** The application can be enhanced with a strong password generator that automatically creates secure passwords containing letters, numbers, and special characters.
6. **Cloud Storage Support:** Future versions may include cloud integration to allow users to access their passwords from multiple devices while maintaining data synchronization.
7. **Backup and Recovery System:** Automatic backup and recovery features can be implemented to prevent data loss and improve reliability.
8. **Mobile Application Development:** The Password Safe system can be converted into an Android or iOS application, enabling users to manage passwords conveniently on mobile devices.
9. **Multi-User Support:** Future versions can support multiple users, allowing each user to maintain a separate and secure password database.

How to Create Strong

P @ S S W O R D S



User-generated passwords should be at least 8 characters



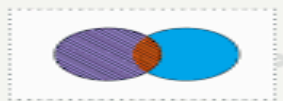
Randomly generated passwords should be at least 6 numeric characters



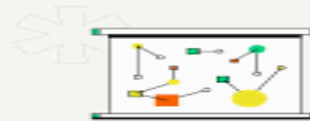
Opt for passphrases that are easier to remember and harder to guess



Check passwords against a list of weak or compromised credentials



Do not reuse passwords across multiple accounts or share passwords via insecure methods like email or chat



Avoid overly complex requirements such as special characters and strict password expiration policies



Integrate tools like password strength meters or strong password generators

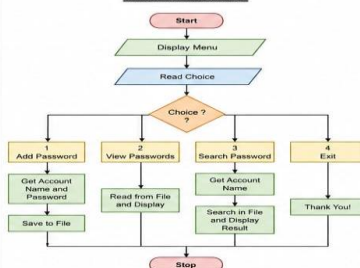


Use password managers for secure storage and sharing



Implement MFA and automatic lockouts for an extra layer of protection

FLOWCHART



CODE SCREENSHOT (VS CODE)

```

1 def add_password():
2     account = input("Enter Account Name: ")
3     password = input("Enter Password: ")
4
5     with open("passwords.txt", "a") as file:
6         file.write(account + "|" + password + "\n")
7
8     print("Password Saved Successfully!")
9
10 def view_passwords():
11     with open("passwords.txt", "r") as file:
12         print(file.read())
13
14 except FileNotFoundError:
15     print("No passwords stored.")
  
```

OUTPUT SCREENSHOTS

1. Add Password

```

PASSWORD SAFE
1. Add Password
2. View Passwords
3. Search Password
4. Exit
Enter Choice: 1
Enter Account Name: Gmail
Enter Password: mygmail123
Password Saved Successfully!
  
```

3. Search Password (Found)

```

PASSWORD SAFE
1. Add Password
2. View Passwords
3. Search Password
4. Exit
Enter Choice: 3
Enter Account Name to Search: Facebook
Password: fbpass456
  
```

PASSWORDS FILE CONTENT (passwords.txt)

```

passwords.txt - Notepad
File Edit Format View Help
Gmail | mygmail123
Facebook | fbpass456
Instagram | insta789
College Mail | college@123
  
```

2. View Passwords

```

PASSWORD SAFE
1. Add Password
2. View Passwords
3. Search Password
4. Exit
Enter Choice: 2
Stored Passwords:
Gmail | mygmail123
Facebook | fbpass456
Instagram | insta789
College Mail | college@123
  
```

3. Search Password (Not Found)

```

PASSWORD SAFE
1. Add Password
2. View Passwords
3. Search Password
4. Exit
Enter Choice: 3
Enter Account Name to Search: Twitter
Account Not Found
  
```

PROGRAM RUN (FULL EXAMPLE)

```

PASSWORD SAFE
1. Add Password
2. View Passwords
3. Search Password
4. Exit
Enter Choice: 1
Enter Account Name: College Mail
Enter Password: college@123
Password Saved Successfully!
Enter Choice: 2
Stored Passwords:
Gmail | mygmail123
Facebook | fbpass456
Instagram | insta789
College Mail | college@123
Enter Choice: 3
Enter Account Name to Search: Instagram
Password: insta789
Enter Choice: 4
Thank You!
  
```

CHAPTER- 10

CONCLUSION

10.1 Conclusion

The Password Safe Using Python project was successfully designed and implemented to provide a simple and efficient password management solution. The application allows users to store, view, and search passwords through a menu-driven interface. The project demonstrates the practical use of Python programming concepts such as functions, loops, conditional statements, and file handling. Through the development of this system, users can manage their account credentials in an organized manner without the need to remember multiple passwords.

The project achieved its primary objective of creating a user-friendly password management application. Testing results showed that all modules functioned correctly and produced the expected outputs. Although the current version uses text-file storage and has limited security features, it provides a strong foundation for future enhancements. Features such as password encryption, graphical user interfaces, database integration, and cloud storage can be added in future versions to improve the security, functionality, and usability of the system.